Practicumopdracht 1

Introductie

Welkom bij het eerste kwantitatieve practicum van M&T I! In dit practicum ga je leren hoe je R kan gebruiken voor data analyse. Je zult jouw eerste stappen maken in het bekijken, prepareren en analyseren van data. Voordat we aan de slag gaan, volgen eerst een aantal instructies.

Als je nog niet in *Visual* mode werkt, klink dan eerst linksboven in jouw scherm op Visual in plaats van Source. Dit werkt een stuk makkelijker.

Voorbereiding

In het quantcrim handboek staat beschreven welke voorbereiding je nodig hebt om deze practicumopdracht te kunnen maken. Dit zijn de volgende dingen:

- Het installeren van R en Rstudio
- Het installeren van de benodigde packages
- Het bestuderen van de stof van week 4 (Data beschrijven)
- Het voltooien van de Codecademy cursus

Als het goed is, heb je deze dingen al gedaan. Mocht dat niet zo zijn, dan moet je hiermee eerst aan de slag. Anders zul je niet in staat zijn het practicum te maken.

Opdracht maken

Je maakt de opdracht in deze zogenoemde .Rmd file. Dit is een bestand waarin je tekst kan typen en R code apart kan uitvoeren. Dat is handig om zowel analyses uit te voeren en daarover te rapporteren. De opdracht zal aangeven wanneer jij aan de beurt bent om iets doen. Dit gaat hoofdzakelijk om twee dingen: het typen van R code en het geven van een antwoord.

R code typ je in een veld zoals hieronder weergeven. Deze blokken waar je code invult, worden ook wel *code chunks* genoemd.

```
# hier voer je R code in
```

Er staan het volgende wanneer je een antwoord moet geven:

> [Typ hier jouw antwoord]

Verder werken we bij de meeste opgaven volgens het principe van *see one, do one, teach one*. Je krijgt eerst een voorbeeld te zien van hoe het moet en daarna moet je het zelf proberen. Als je daarin geslaagd bent, kun je eventueel een medestudent helpen. Iets doceren is de beste manier om te leren!

Een ander belangrijk principe is dat je zo min mogelijk code kopieert en plakt, maar het juist zelf typt. Typen is een actieve handeling terwijl kopiëren/plakken passief is. Je zult R veel sneller leren als je zelf actief typt en daarbij nadenkt over wat hetgene betekent dat je aan typen bent.

Op het eind van de opdracht maak je een pdf van dit bestand en lever je dit in via Canvas om aan de **inspanningsverplichting** te voldoen (**deadline: vrijdag 23:59**). Instructies daarvoor staan op het eind.

Data laden

Aan het begin van een analyse moeten we altijd als eerste de relevante packages laden. Die staan hieronder weergeven in het codeblok. Als het goed is, heb je deze al vooraf geïnstalleerd (zie Installatie). De code zal niet werken als je dit niet hebt gedaan.

Je kunt de code selecteren en 'runnen' (Ctrl + Enter) of je klikt op het driehoekje rechtsboven in het codeblok. Dit runnen zul je bij veel codebloks moeten doen. Let dus goed op hoe het moet.

Eventuele waarschuwingen in de output kun je hier negeren.

```
# laden packages
library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
## Warning: package 'dplyr' was built under R version 4.4.3
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(modeest)
library(plotrix)
```

```
## Warning: package 'plotrix' was built under R version 4.4.3
```

```
library(rmarkdown)
```

```
## Warning: package 'rmarkdown' was built under R version 4.4.2
```

De volgende stap is de dataset te laden die we gebruiken voor onze analyses. Dit is de synthetische dataset gebaseerd op Het Grote Uitgaansonderzoek, zoals beschreven onder Data. Daarvoor moet je eerst de dataset data_uitgaan.rds downloaden via Opdrachten als je dat nog niet gedaan hebt. Sla deze dataset op in je Bureaublad.

Hieronder staat voorbeeldcode van hoe je data laadt in R. Deze code hoef je niet te runnen (zal ook niet werken).

```
# laden data
data_uitgaan <- read_rds("C:/Users/gijs_/Desktop/data_uitgaan.rds")
```

Wat zie je hierboven?

`# laden data` is een beschrijving van wat de regel daaronder doet. We gebruiken hashtags in R om toelichting te geven over wat een stukje code doet. Zoals je hebt gezien in het quantcrim handboek, kan dit zowel boven als naast de code worden gezet.

`data_uitgaan` is de naam die we het aan dataframe geven. Die is in dit geval hetzelfde als de externe dataset.

`read_rds()` is de functie om de externe dataset `data_uitgaan.rds` te laden. Je zou deze functie kunnen zien als "lees een .rds bestandstype in"

`"C:/Users/gijs_/Desktop/data_uitgaan.rds"` is de locatie waar de externe dataset is opgeslagen op mijn persoonlijke computer. R weet dan waar de externe dataset zich bevindt. Je kunt zien dat ik het bestand op

mijn Bureaublad (Desktop) heb opgeslagen.

`<-` zorgt ervoor dat R de externe dataset opslaat in een dataframe object die we `data_uitgaan` noemen. Dit noemen we de *assignment operator*.

Nu kun je de externe dataset zelf gaan laden. Zorg ervoor dat je de data hebt gedownload en opgeslagen op je Bureaublad. Pas vervolgens de code zo aan dat R de juiste locatie vindt van het bestand. Dit doe je door het juiste pad tussen aanhalingstekens aan de te passen (`"Hier_jouw_pad/data_uitgaan.rds"`). Je kunt op je computer naar de locatie van het bestand gaan (Bureaublad) en dan via rechtermuisknop en 'Als pad kopiëren' de exacte locatie van het bestand vinden en hier plakken. Let op dat je in R `/` gebruikt voor bestandslocaties in plaats van \. Dit moet je nog verder aanpassen. Zorg ervoor dat jouw code het voorbeeld volgt van het codeblok hierboven. Run vervolgens de code.

1. (Als je er echt niet uitkomt, kun je ook op het bestand van de externe dataset dubbelklikken en dit direct laden in R. Je krijg dan een pop-up met de vraag of je dit bestand direct wil laden in R. Klik op OK)
2. (Je kunt ook altijd Googlen of DuckDuckGo om uit te vinden hoe iets moet)

```
# laden data
data_uitgaan <- read_rds("Hier_jouw_pad/data_uitgaan.rds")
```

```
## Warning in readRDS(con, refhook = refhook): cannot open file
## 'Hier_jouw_pad/data_uitgaan.rds': No such file or directory
```

```
## Error in readRDS(con, refhook = refhook): cannot open the connection
```

Als het is gelukt, zul je rechtsboven in Rstudio een regel zien met `data_uitgaan` onder het tabblad Environment. Daar zie je ook hoeveel observaties en variabelen de dataset heeft.

Data inspecteren

De volgende stap na het laden van de data, is het inspecteren ervan. Hierdoor krijgen we een globaal idee van hoe onze dataset in elkaar zit. Hiervoor gebruiken we de functie `View()`. R opent dan het dataframe en je ziet de observaties en variabelen. Run onderstaande code. Scroll een beetje van onder naar boven en links en rechts en denk na over wat de waardes die je ziet kunnen betekenen.

```
# inspecteren van de data
View(data_uitgaan)
```

Variabelen beschrijven

Een eerste stap in analyse is dat we een beschrijving geven van de variabelen waarmee we werken. In dit practicum werken we met de volgende variabelen (en antwoordcategorieen):

`opleidingsniveau`: Het hoogst behaalde opleidingsniveau van de respondent

- laag / midden / hoog

`cannabis_ooit`: Of de respondent ooit cannabis (wiet of hasj) heeft gebruikt.

- ja / nee

`cannabis_frequentie`: frequentie waarmee respondent cannabis (wiet of hasj) gebruikt.

- één keer / minder dan maandelijks / eens per maand / een paar keer per maand / eens per week / een paar keer per week / (bijna) elke dag

`gevoelstemperatuur_cannabis`: welk gevoel een respondent bij cannabis in het algemeen heeft (de beoordeling ervan).

- score van 0 tot en met 100, waarbij 0 staat voor negatief en 100 voor positief.

Bedenk eerst wat het meetniveau is van deze variabelen. Geef je antwoord met korte uitleg hieronder:

`opleidingsniveau`: [Jouw antwoord hier]

`cannabis_ooit`: [Jouw antwoord hier]

`cannabis_frequentie`: [Jouw antwoord hier]

`gevoelstemperatuur_cannabis`: [Jouw antwoord hier]

Eenvoudige tabel

Met aantallen We gaan nu voor opleidingsniveau kijken wat de verdeling is op deze variabele. Dit doen we eerst middels een tabel. Daarvoor gebruiken we de functie `table()`. Hieronder zie je een voorbeeld voor de variabele `geslacht`.

Je vraagt hier een tabel op van een variabele via `table()`. Om die variabele te selecteren, moet je eerst het dataframe aangeven: `data_uitgaan`. Binnen dat dataframe geef je aan om welke variabele het gaat. Dit doe je middels de `$` operator. Deze zegt het ware: binnen `data_uitgaan`, neem de variabele `geslacht`: `data_uitgaan$geslacht`. Run nu de code hieronder.

```
# tabel van geslacht met aantallen
table(data_uitgaan$geslacht)
```

```
##
##    man  vrouw anders
##    178    170      2
```

Als je dit codeblok runt, zul je net als bij Meetniveas en visualisaties zien dat in de dataset 178 mannen, 170 vrouwen en 2 anders-identificerenden zitten.

Je zou nu zelf in staat moeten zijn een tabel voor `opleidingsniveau` te maken. Doe dat in het codeblok hieronder. Typ de code zelf hier onder `# tabel van opleidingsniveau met aantallen`. Vergeet niet de code te runnen.

```
# tabel van opleidingsniveau met aantallen
```

Als het is gelukt de tabel te maken, rapporteer dan hoeveel respondenten er voor elke categorie in de dataset zitten.

[Jouw antwoord hier]

Met proporties Als je de proporties van elke categorie van een variabele wil weten, kun je expliciet aangeven dat je proporties wil berekenen. Dit doe je door `|> prop.table()` toe te voegen. `prop.table()` is een functie om de proporties binnen een tabel te berekenen.

`|>` is een operator die je nog vaker zult tegenkomen. Het neemt een stuk code aan de linkerkant van de operator en geeft het als eerste argument aan de rechterkant. Dit wil zeggen dat `table(data_uitgaan$geslacht) |> prop.table()` hetzelfde is als `prop.table(table(data_uitgaan$geslacht))`. Als je dit nu nog niet snapt, is dat helemaal niet erg. Je zult gedurende de cursus gaan zien waarom het meerwaarde heeft om R code zo te schrijven. Hieronder staat een voorbeeld met de variabele `geslacht`. Run deze code.

```
# tabel van geslacht met proporties
table(data_uitgaan$geslacht) |> prop.table()
```

```
##
##         man       vrouw      anders
## 0.508571429 0.485714286 0.005714286
```

De output geeft nu proporties weer. De dataset bestaat dus (afgerond) uit 50,9% mannen, 48,6% vrouwen en 0,6% anders-identificerenden.

Maak nu een tabel van opleidingsniveau met proporties. Op basis van bovenstaande instructies kun je uitvogelen hoe dat moet. Typ de juiste code onder `# tabel van opleidingsniveau met proportiesen` run vervolgens de code.

```
# tabel van opleidingsniveau met proporties
```

Rapporteer nu de proporties voor elke categorie van de variabele `opleidingsniveau`.

[Jouw antwoord hier]

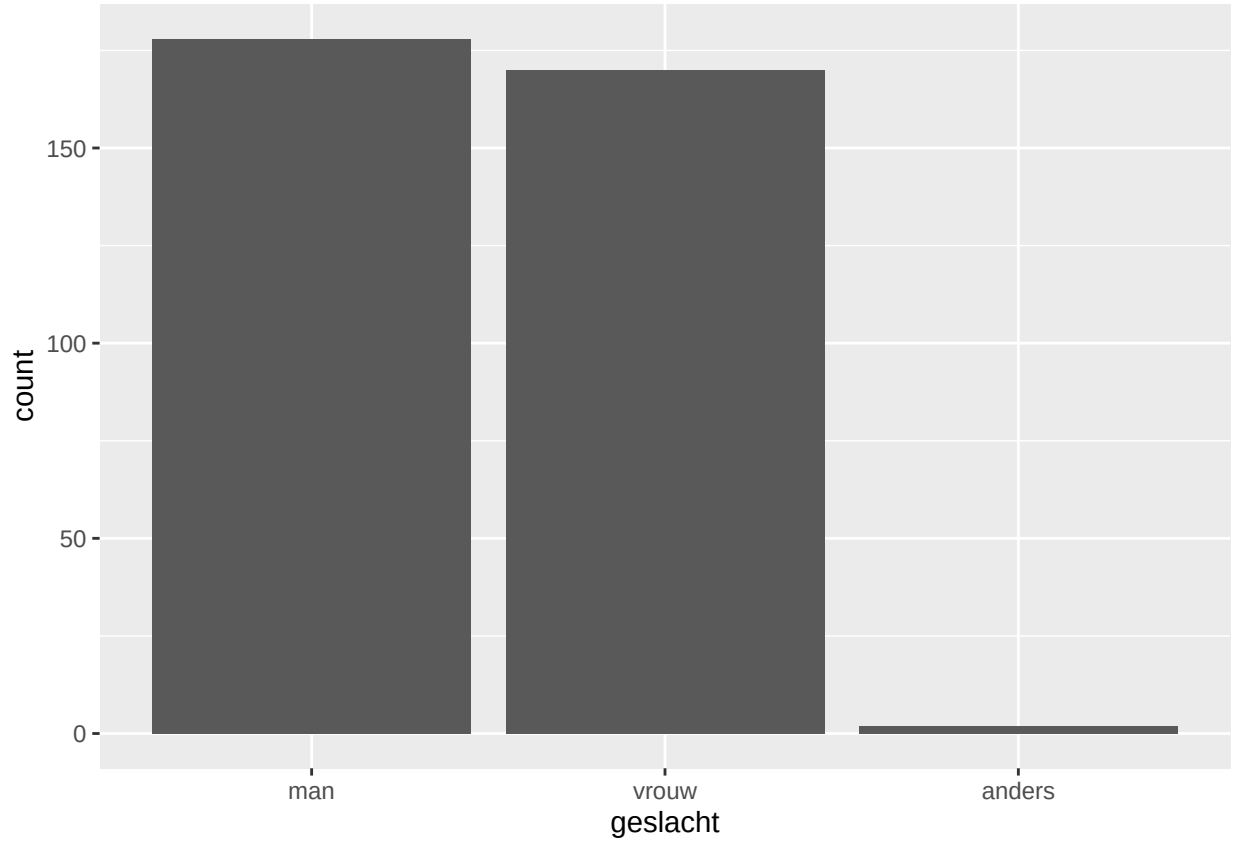
Staafdiagram

We gaan nu een staafdiagram maken van `opleidingsniveau`. De basis van datavisualisatie (het maken van figuren) is de functie `ggplot()`. Deze functie kunnen we, zoals je zult zien, uitbouwen met diverse componenten waardoor we figuren laag voor laag kunnen vormgeven.

De basis die we vaak gebruiken is `ggplot(data, aes(x = variabele))`. Hier moeten we bij `data` de naam van onze dataset invoeren en bij `variabele` de naam van onze variabele die we willen visualiseren. Vervolgens voegen we een geometrische laag (een type figuur) toe die aangeeft wat voor soort visualisatie het moet worden (bijvoorbeeld staafdiagram, puntenwolk, histogram etc.). We beginnen hier met een staafdiagram, dit gaat via `geom_bar()`. Voor `ggplot()` geldt dat we elke laag toevoegen middels een `+`-teken. Zo kunnen we een plot/visualisatie laag voor laag opbouwen.

Hieronder zie je weer het voorbeeld van de variabele `geslacht`. We hebben tevens de naam onze dataset ingevuld: `data_uitgaan`. Run de code.

```
# staafdiagram van geslacht
ggplot(data_uitgaan, aes(x = geslacht)) +
  geom_bar()
```



Net als in het quantcrim handboek, zie je hier een basis staafdiagram van geslacht.

Maak nu een staafdiagram van `opleidingsniveau`. Typ de juiste code in het codeblok hieronder en run de code.

```
# staafdiagram van opleidingsniveau
```

We gaan nu kijken hoe je een staafdiagram kan 'oppimpen'. ggplot biedt veel opties om figuren op een meer aantrekkelijke en duidelijke manier te presenteren. We gaan hier de achtergrond, de assenlabels en titel aanpassen.

`theme_bw()` staat voor 'thema in zwart-wit (black and white)'. Dit zorgt ervoor dat de achtergrond wit wordt en de staven zwart.

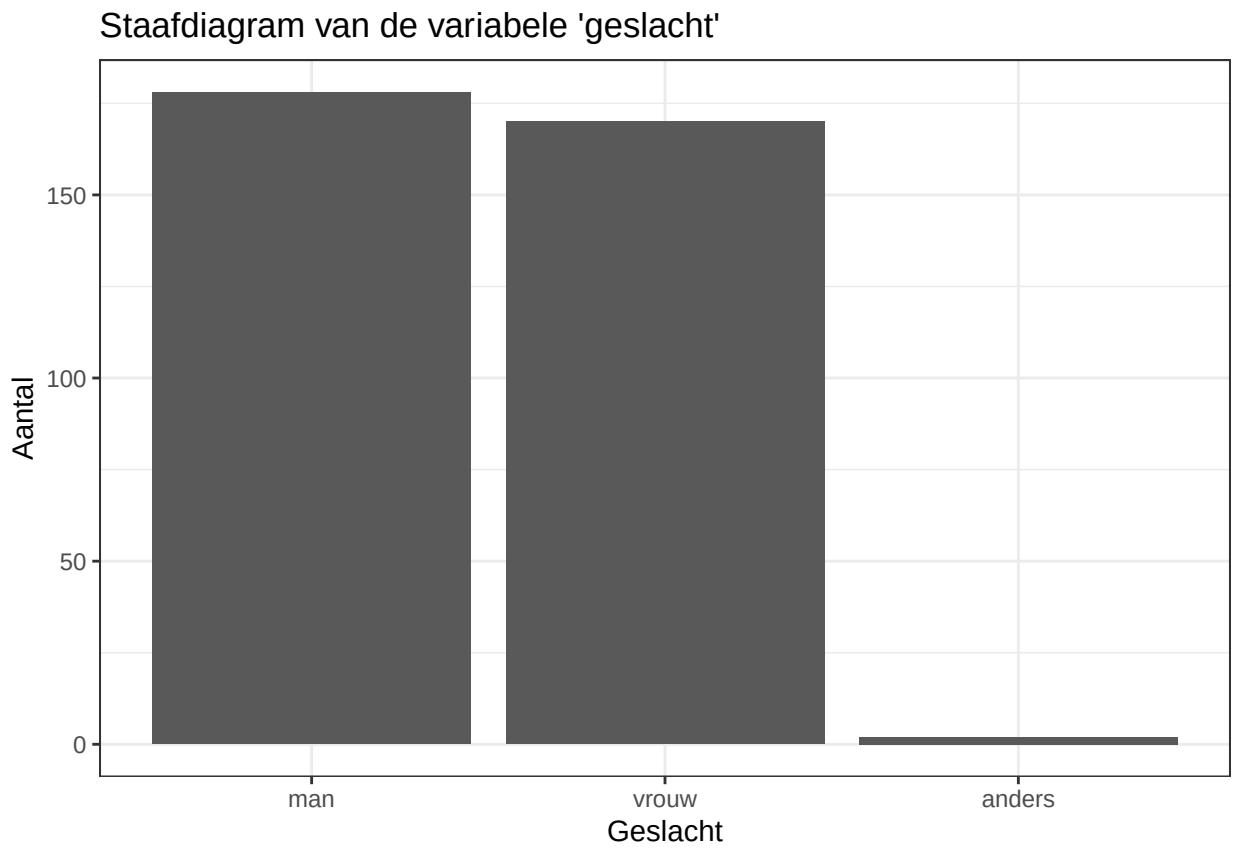
`ylab()` past het label van de y-as aan. Je moet het nieuwe label tussen aanhelingstekens zetten: `ylab("Aantal")`.

`xlab()` werkt hetzelfde als `ylab()`, maar dan voor de x-as: `xlab("Geslacht")`

`ggtitle()` geeft een titel aan de figuur, ook hier moet de tekst tussen aanhalingstekens: `ggtitle("Staafdiagram van de variabele 'geslacht'")` .

Met behulp van het `+`-teken voegen we alle lagen toe aan de figuur van geslacht. Run de code.

```
# staafdiagram van geslacht met opmaak
ggplot(data_uitgaan, aes(x = geslacht)) +
  geom_bar() +
  theme_bw() +
  ylab("Aantal") +
  xlab("Geslacht") +
  ggtitle("Staafdiagram van de variabele 'geslacht'")
```



Dat ziet er al een stuk beter uit. Vergelijk deze opgemaakte staafdiagram met de basis staafdiagram van geslacht en kijk welke veranderingen er plaats hebben gevonden.

Pas nu de staafdiagram op dezelfde manier aan voor `opleidingsniveau`. Verander de achtergrond, labels van de assen en titel. Zorg dat deze passend zijn voor deze variabele. Breid de code uit die je eerder hebt gemaakt.

```
# staafdiagram van opleidingsniveau met opmaak
```

Variabelen hercoderen

```
summary(data_uitgaan$gevoelstemperatuur_cannabis)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##    0.00   43.00   55.00   55.94   70.00  100.00      12
```

Vooraf: 1) leren met R 2) voorbereiding 3) typen in R (J Roos) 4) manier van werken (antwoord geven, r chunks) 5) inleveren op einde

- Introductie data
- Verkennen data
  - `summary, table`
  - simpele visualisatie
  - opbouwen opmaak visualisatie
  - kort Beschrijven

    - hercoderen, missende waarden
- Relatie tussen data.

```
View(data_uitgaan)
```